

Programação Concorrente

Exame de Recurso¹

4 de junho de 2024

Duração: 2h00m

I

- 1 Se estivermos a usar monitores, podemos ser tentados a resolver qualquer problema de sincronização com apenas uma variável de condição. Explique que problemas tem tal abordagem, e dê um exemplo em que será preferível usar mais do que uma variável de condição.
- 2 Explique porque em Erlang, para esperar por uma mensagem de resposta a um pedido, não deve ser utilizado `receive X -> ...`, e diga como deve ser feito. Dê um exemplo, relativamente a uma operação que devolve uma resposta, descrevendo o protocolo e mostrando o código de uma função de interface.

II

Implemente em Java, fazendo uso de primitivas baseadas em monitores, uma interface `Queue`, para uso concorrente, com as seguintes operações:

```
interface Queue {  
    void enqueue(int e);  
    int[] dequeue(int n) throws InterruptedException;  
}
```

Em `enqueue(e)` acrescenta o elemento `e` no fim da queue, e `dequeue(n)` remove e devolve os primeiros `n` elementos atomicamente, devendo bloquear a thread invocadora até tal ser possível. O construtor deve ter um parâmetro `m`, que define o número máximo de elementos que pode ser removido simultaneamente, i.e., $0 < n \leq m$. Considere ainda que, caso estejam várias threads à espera, deve ser dada prioridade a operações que requisitem mais elementos; por exemplo, `dequeue(5)` deverá ter prioridade sobre `dequeue(4)`, fazendo com que esta última continue à espera quando é inserido um quarto elemento.

III

Apresente o código Erlang para implementar o mesmo serviço descrito no grupo II, através de um ou mais processos. Suponha que os clientes são processos Erlang que comunicam pelo mecanismo nativo de mensagens, e implemente também as funções de interface apropriadas para serem usadas por estes para interagirem com o(s) processo(s) relevante(s).